

RESOLUCIÓN DE MODELOS DE PROGRAMACIÓN LINEAL MEDIANTE MÉTODOS GRÁFICO Y SIMPLEX (Agosto de 2026)

Martin Pineda Jaramillo

Facultad de Ciencias Básicas e Ingeniería, Universidad de los Llanos, Villavicencio, Colombia

m.pineda@unillanos.edu.co, 160005227

Resumen

Este documento presenta el diseño y desarrollo de una aplicación interactiva construida en Python para la resolución de modelos de programación lineal. La herramienta consolida las soluciones mediante el método gráfico para problemas de dos variables y el método Simplex algorítmico para modelos de mayor complejidad. El diseño del sistema se ha estructurado mediante Historias de Usuario, y abstraído a nivel general a través de diagramas UML de clases y secuencias. El despliegue unificado permite a los estudiantes de la asignatura de Optimización visualizar las regiones factibles y comprender iterativamente el cálculo de pivotes en los poliedros n-dimensionales.

Palabras clave: Método Gráfico, Método Simplex, Optimización, Programación Lineal, Python,

I. INTRODUCCIÓN

La Programación Lineal es una de las herramientas más utilizadas dentro de la investigación de operaciones, ya que permite modelar y resolver problemas de asignación óptima de recursos limitados, sujetos a un conjunto de restricciones lineales. Su aplicación abarca distintas áreas como la producción industrial, la logística, la economía y la administración de proyectos, en donde se busca maximizar beneficios o minimizar costos.

Resolver manualmente estos problemas, especialmente cuando el número de variables y restricciones crece, resulta un proceso tedioso y propenso a errores de cálculo. Por esta razón, resulta de gran utilidad contar con herramientas informáticas capaces de automatizar dicho proceso, tanto con fines académicos como prácticos.

El presente trabajo tiene como objetivo desarrollar una aplicación de escritorio que permita resolver problemas de Programación Lineal utilizando dos de los métodos más representativos: el Método Gráfico, aplicable únicamente a problemas de dos variables, y el Método Simplex, aplicable a problemas de n variables. Además, se documenta el proceso de análisis del software mediante historias de usuario, y su diseño mediante diagrama de clases y diagrama de secuencia, siguiendo buenas prácticas de ingeniería de software.

II. MARCO TEÓRICO

2.1 Programación Lineal:

La Programación Lineal es una técnica matemática de optimización que consiste en maximizar o minimizar una función lineal, denominada función objetivo, sujeta a un conjunto de restricciones también lineales, expresadas como ecuaciones o inecuaciones. Todo problema de programación lineal está compuesto

Solución de Problemas de Programación Lineal

por tres elementos principales: la función objetivo, las variables de decisión y las restricciones, incluyendo la condición de no negatividad de las variables.

2.1 Método Gráfico:

El método gráfico es un procedimiento de solución aplicable exclusivamente a problemas con dos variables de decisión. Consiste en representar cada restricción como una recta en el plano cartesiano, determinar la región factible como la intersección de los semiplanos definidos por las restricciones, e identificar los vértices de dicha región. Posteriormente, se evalúa la función objetivo en cada uno de los vértices factibles, y el valor óptimo corresponde al vértice que maximiza o minimiza dicha función, según corresponda.

2.1.1 Ejemplo Matemático:

Para ilustrar el procedimiento, considere el siguiente problema clásico:

$$\text{Max } Z = 3x_1 + 5x_2$$

Sujeto a:

$$x_1 \leq 4$$

$$2x_2 \leq 12$$

$$3x_1 + 2x_2 \leq 18$$

$$x_1, x_2 \geq 0$$

Al graficar las tres restricciones junto con los ejes, se obtienen los siguientes vértices de la región factible: (0,0), (4,0), (4,3), (2,6) y (0,6). Evaluando Z en cada uno:

$$Z(0,0) = 0$$

$$Z(4,0) = 12$$

$$Z(4,3) = 27$$

$$Z(2,6) = 36$$

$$Z(0,6) = 30$$

El valor óptimo es $Z = 36$, alcanzado en el punto (2,6). Este es el mismo procedimiento que reproduce internamente la clase GraphicSolver: calcula las intersecciones entre restricciones, filtra las que son factibles y evalúa la función objetivo en cada una.

2.2 Método Simplex:

El método Simplex, propuesto por George Dantzig, es un algoritmo iterativo utilizado para resolver problemas de programación lineal con cualquier número de variables. El método parte de una solución básica factible inicial, representada en una tabla simplex, e itera mejorando dicha solución mediante operaciones de pivoteo, hasta alcanzar la solución óptima, momento en el cual todos los coeficientes de la fila correspondiente a la función objetivo son mayores o iguales a cero. En cada iteración se identifica la variable entrante mediante el coeficiente más negativo de la fila objetivo, y la variable saliente mediante el criterio de la razón mínima entre los términos independientes y los coeficientes de la columna pivote.

Solución de Problemas de Programación Lineal

2.3 Ingeniería de software:

Las historias de usuario son una técnica ágil utilizada para describir de manera breve y desde la perspectiva del usuario final, una funcionalidad que el sistema debe cumplir. El diagrama de clases es un diagrama estructural de UML que representa las clases del sistema, sus atributos, métodos y las relaciones entre ellas. El diagrama de secuencia, por su parte, es un diagrama de comportamiento de UML que muestra la interacción entre los objetos del sistema a lo largo del tiempo, evidenciando el orden en que se envían los mensajes entre ellos.

III. DESARROLLO DE CONTENIDOS

3.1 Historias de usuario:

ID	Título	Descripción	Criterio de aceptación
HU-01	Configuración de variables y restricciones	Como usuario, quiero ingresar el número de variables y restricciones, para definir el tamaño de mi problema.	Al ingresar los valores y confirmar, el sistema genera los campos correspondientes.
HU-02	Selección del método de resolución	Como usuario, quiero elegir entre método Gráfico o Simplex, para resolver el problema como lo necesito.	Al seleccionar un método y resolver, el sistema aplica la lógica correspondiente.
HU-03	Ingreso de la función objetivo y las restricciones	Como usuario, quiero ingresar la función objetivo y las restricciones con su operador y valor, para definir completamente mi problema.	Al completar los campos y resolver, el sistema arma correctamente el problema con esos datos.
HU-04	Selección del tipo de optimización	Como usuario, quiero elegir entre maximizar o minimizar, para adaptar la solución a mi caso.	Al elegir un tipo de optimización, el sistema lo aplica; si es Simplex y se elige minimizar, muestra un aviso.
HU-05	Visualización de la solución obtenida	Como usuario, quiero ver el resultado del problema, ya sea como gráfico o como tablas de iteraciones, para entender cómo se llegó a la respuesta.	Al finalizar la resolución, el sistema muestra el valor óptimo y su representación (gráfico o tablas), o un mensaje de error si no hay solución.

Tabla 1. Historias de usuario.

3.2 Arquitectura general del sistema:

El sistema sigue un enfoque orientado a objetos. Una clase base abstracta, LinearSolver, almacena los atributos comunes a cualquier problema de PL: función objetivo, restricciones y tipo de optimización. De ella heredan GraphicSolver y SimplexSolver, cada una con su propia implementación del método resolver(). La interfaz gráfica, gestionada por la clase LinearProgrammingApp, construye los formularios, invoca al solver correspondiente y despliega los resultados.

Solución de Problemas de Programación Lineal

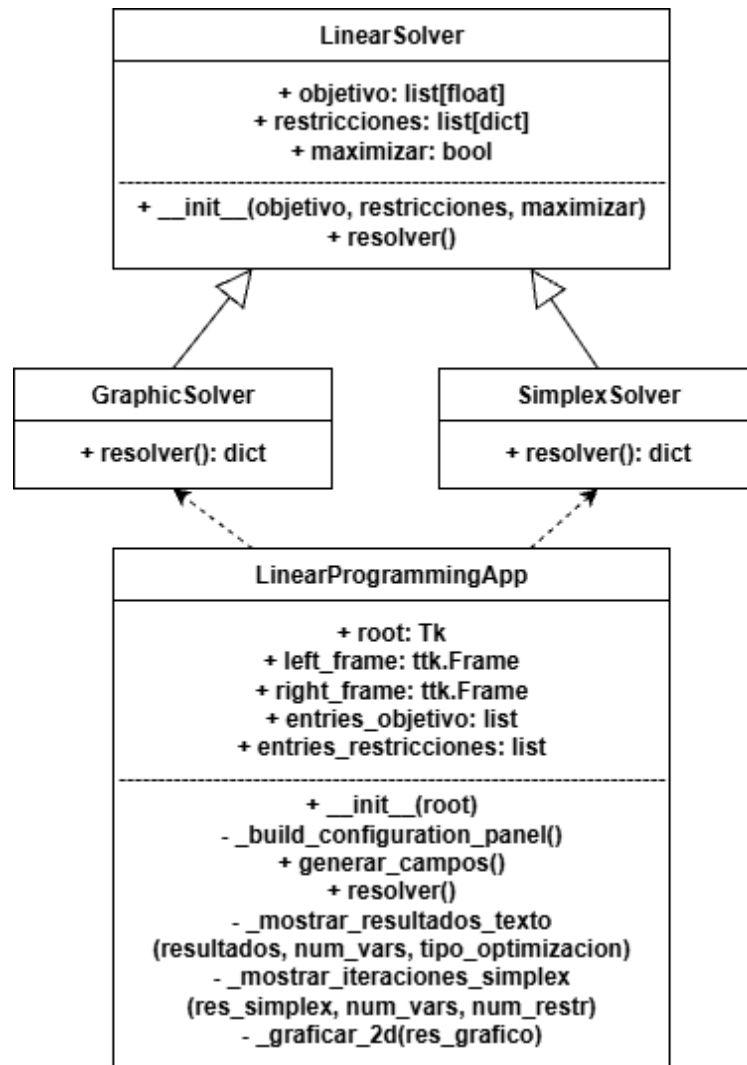


Figura 1. Diagrama de clases.

3.3 Diagrama de secuencia:

El diagrama de secuencia documenta el flujo entre el usuario, la interfaz (**LinearProgrammingApp**) y el solver correspondiente. El usuario configura el problema y presiona "Resolver Problema"; la interfaz recolecta los datos, instancia el solver adecuado, este ejecuta `resolver()` y retorna un diccionario con los resultados, que finalmente la interfaz despliega como texto, tabla de iteraciones o gráfico.

Solución de Problemas de Programación Lineal

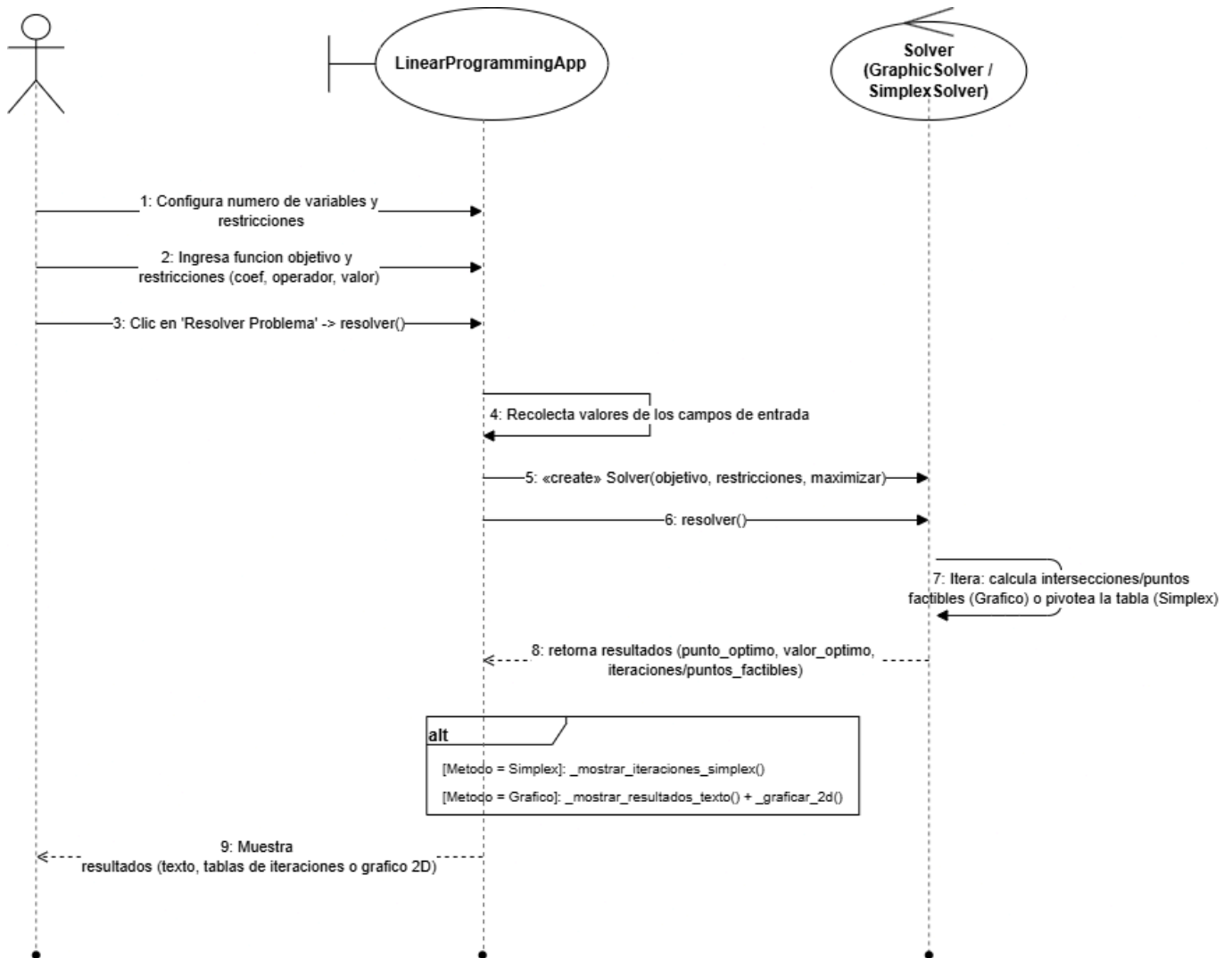


Figura 2. Diagrama de secuencia.

3.3 Implementación del método gráfico:

La clase `GraphicSolver` calcula los puntos de intersección entre cada par de restricciones (incluyendo los ejes) y conserva únicamente los que cumplen todas las condiciones del problema, es decir, los vértices factibles:

Solución de Problemas de Programación Lineal

```
1 for i in range(cant_restricciones):
2     for j in range(i + 1, cant_restricciones):
3         A = np.array([restricciones_completas[i]["coef"], restricciones_completas[j]["coef"]], dtype=float)
4         b = np.array([restricciones_completas[i]["val"], restricciones_completas[j]["val"]], dtype=float)
5
6         if np.linalg.matrix_rank(A) == 2:
7             pt = np.linalg.solve(A, b)
8             puntos.append(pt)
9
10    puntos_factibles = []
11    for pt in puntos:
12        x1, x2 = pt
13        if x1 < -1e-9 or x2 < -1e-9:
14            continue
15
16        valid = True
17        for r in restricciones_completas:
18            val = r["coef"][0] * x1 + r["coef"][1] * x2
19            op = r["op"]
20            target = r["val"]
21
22            if op == "<=" and val > target + 1e-9:
23                valid = False; break
24            elif op == ">=" and val < target - 1e-9:
25                valid = False; break
26            elif op == "=" and abs(val - target) > 1e-9:
27                valid = False; break
28
29        if valid:
30            puntos_factibles.append(pt)
```

Código 1. Cálculo de intersecciones y filtrado de puntos factibles.

Una vez obtenidos los vértices factibles, se evalúa la función objetivo en cada uno y se selecciona el máximo o el mínimo según corresponda, tal como en el ejemplo matemático mostrado anteriormente:

```
1 for p in puntos_factibles:
2     z = self.objetivo[0] * p[0] + self.objetivo[1] * p[1]
3     z_valores.append(z)
4
5     if self.maximizar:
6         indice_optimo = np.argmax(z_valores)
7     else:
8         indice_optimo = np.argmin(z_valores)
9
10    punto_optimo = puntos_factibles[indice_optimo]
11    valor_z_optimo = z_valores[indice_optimo]
```

Código 2. Evaluación en la función objetivo.

3.3 Implementación del Método Simplex:

Solución de Problemas de Programación Lineal

El núcleo del algoritmo simplex es un bucle que, en cada iteración, identifica la columna pivote (coeficiente más negativo de la fila objetivo), la fila pivote (criterio de razón mínima) y realiza el pivoteo sobre toda la tabla:

```
1 while True:
2     fila_z = tabla[-1, :-1]
3
4     if np.all(fila_z >= -1e-9):
5         break
6
7     col_pivote = np.argmin(fila_z)
8
9     col_valores = tabla[:-1, col_pivote]
10    lados_derechos = tabla[-1, -1]
11
12    cocientes = []
13    for i in range(num_restr):
14        if col_valores[i] > 1e-9:
15            cocientes.append(lados_derechos[i] / col_valores[i])
16        else:
17            cocientes.append(np.inf)
18
19    fila_pivote = np.argmin(cocientes)
20
21    if cocientes[fila_pivote] == np.inf:
22        return None
23
24    elemento_pivote = tabla[fila_pivote, col_pivote]
25
26    tabla[fila_pivote, :] /= elemento_pivote
27
28    for i in range(len(tabla)):
29        if i != fila_pivote:
30            factor = tabla[i, col_pivote]
31            tabla[i, :] -= factor * tabla[fila_pivote, :]
32
33    iteraciones.append(tabla.copy())
```

Código 3. Bucle principal del método Simplex

Este proceso se repite hasta que la fila objetivo no tenga coeficientes negativos, momento en que la tabla contiene la solución óptima. Finalmente, el sistema recorre las columnas de la tabla para identificar las variables básicas y extraer su valor:

```
1 for j in range(num_vars):
2     columna = tabla[:, j]
3     es_uno = np.isclose(columna, 1.0)
4     es_cero = np.isclose(columna, 0.0)
5
6     if np.count_nonzero(es_uno) == 1 and np.count_nonzero(es_cero) == len(columna) - 1:
7         fila_uno = np.where(es_uno)[0][0]
8         if fila_uno < num_restr:
9             solucion_vars[j] = tabla[fila_uno, -1]
10
11 valor_z_optimo = tabla[-1, -1]
```

Código 4. Interpretación de la tabla final

3.4 Interfaz de usuario:

Solución de Problemas de Programación Lineal

La interfaz se organiza en un panel izquierdo de configuración (número de variables, restricciones, método y tipo de optimización) y un panel derecho de resultados, donde se muestra la solución en texto, tabla de iteraciones o gráfico bidimensional incrustado mediante FigureCanvasTkAgg.

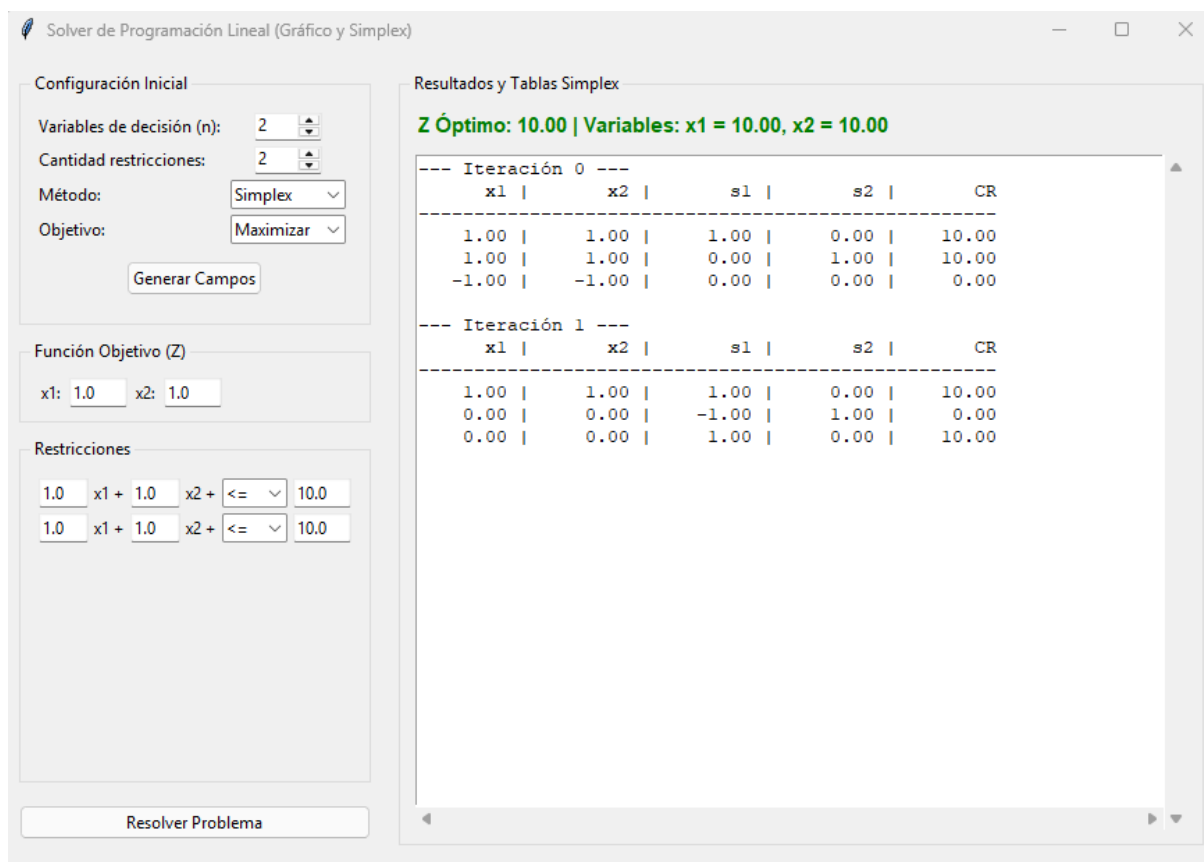


Figura 3. Interfaz gráfica principal del sistema

IV. RESULTADOS

El sistema fue probado con distintos problemas de dos y más variables. En los casos de dos variables, el método gráfico identificó correctamente la región factible, sus vértices y el punto óptimo, coincidiendo con el resultado calculado manualmente en el ejemplo matemático presentado ($Z = 36$ en el punto $(2,6)$). En los casos resueltos por simplex, el sistema construyó correctamente la tabla inicial, ejecutó las iteraciones necesarias y llegó a la solución óptima, mostrando el detalle de cada tabla intermedia. El sistema también respondió correctamente ante datos inválidos, problemas no acotados y problemas sin región factible.

Solución de Problemas de Programación Lineal

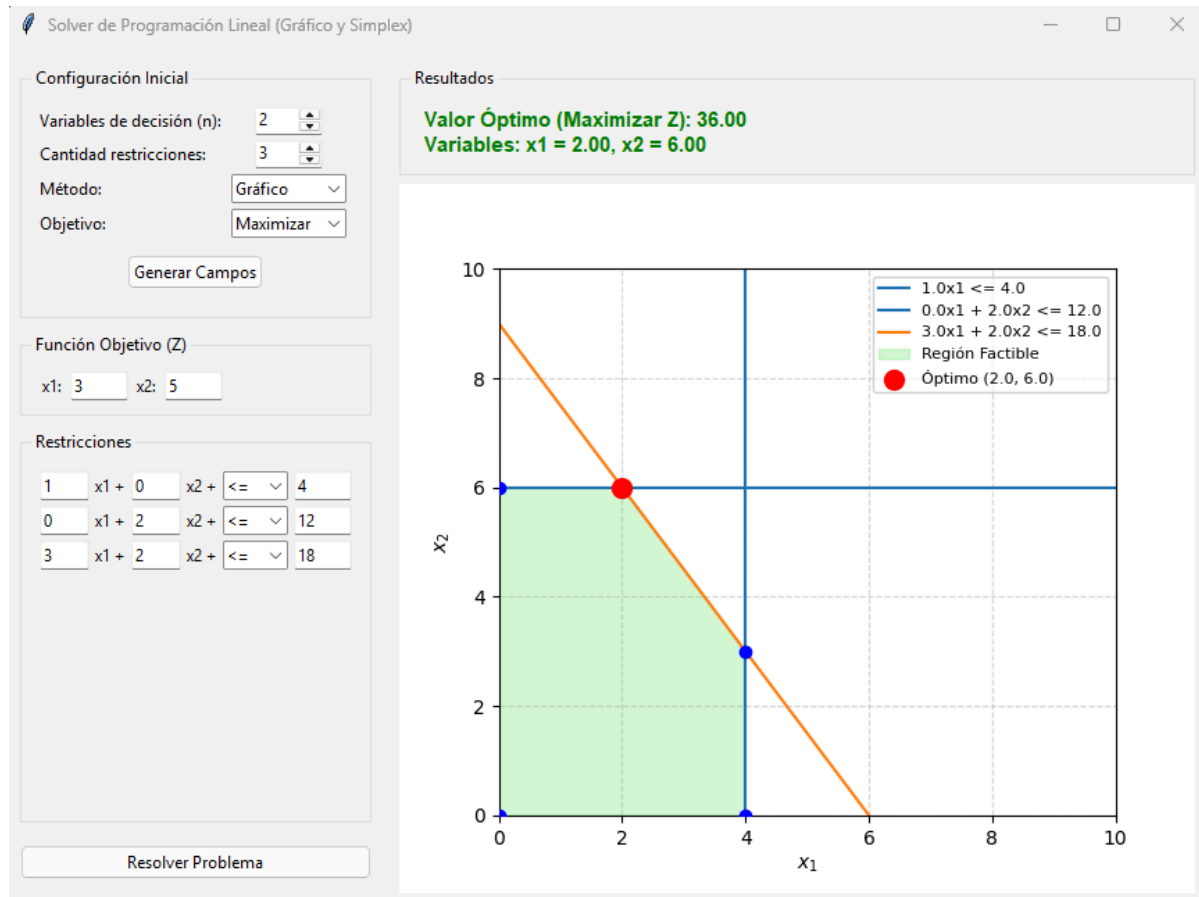


Figura 4. Resultado obtenidos para el ejemplo por método gráfico

Solución de Problemas de Programación Lineal

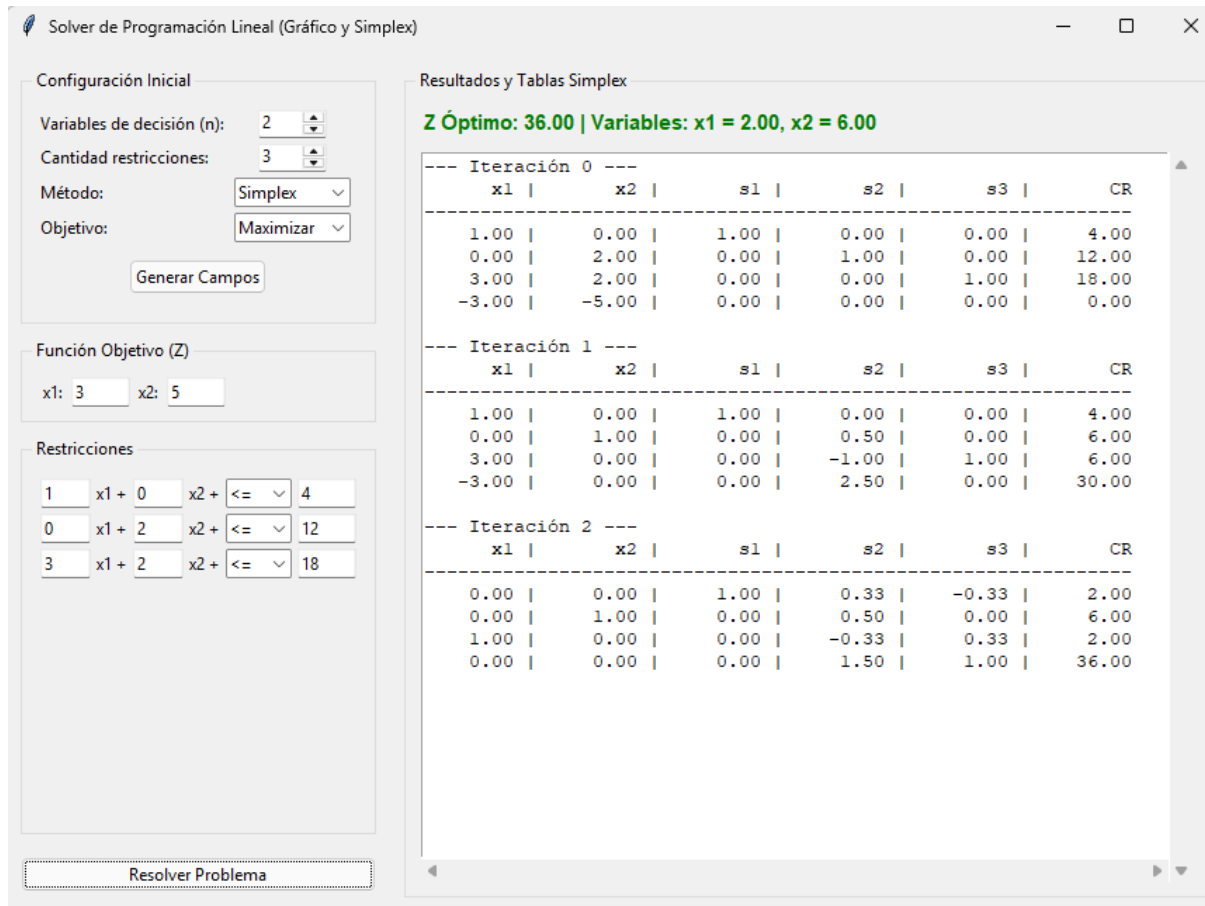


Figura 5. Resultado obtenidos para el ejemplo por método simplex

V. CONCLUSIONES

El desarrollo del programa permitió comprobar el funcionamiento del método gráfico y del método simplex, evidenciando sus diferencias y limitaciones, principalmente en el número de variables que cada uno puede manejar. La arquitectura orientada a objetos, basada en la herencia desde LinearSolver, facilitó que ambos métodos compartieran una estructura común. El uso de historias de usuario, diagrama de clases y diagrama de secuencia resultó clave para definir los requerimientos y la interacción entre componentes antes de programar. Como trabajo futuro, se podría extender el método gráfico para casos no acotados y permitir que el simplex resuelva minimización sin transformación previa por parte del usuario.

VI. REFERENCIAS

- [1] G. B. Dantzig, Linear Programming and Extensions. Princeton, NJ, USA: Princeton University Press, 1963.
- [2] H. A. Taha, Investigación de Operaciones, 9na ed. México: Pearson Educación, 2012.
- [3] Preparación de los documentos IEEE Transaction and Journals, diciembre, 2020.
- [4] F. S. Hillier y G. J. Lieberman, Introducción a la Investigación de Operaciones, 10ma ed. México: McGraw-Hill, 2015.